

Autoenkoderi za bežične komunikacije: end-to-end naučeni komunikacioni sistem

Nikola Bakić, EE58/2022, FTN Novi Sad
jun 2026. god.

1 Uvod

Klasičan digitalni komunikacioni sistem gradi se kao lanac nezavisno projektovanih blokova: koder izvora, koder kanala, modulator, kanal, demodulator i odgovarajući dekoderi na prijemu. Svaki blok optimizuje se zasebno, prema sopstvenom kriterijumu, što garantuje da je pojedinačno (lokalno) optimalan, ali ne i da je ceo sistem zajednički optimizovan za krajnji cilj, pouzdan prenos informacije uz minimalnu verovatnoću greške.

Autoenkoderski pristup fizičkom sloju, koji su uveli O'Shea i Hoydis [1], posmatra ceo komunikacioni lanac kao jednu neuronsku mrežu tipa autoenkoder. Predajnik (enkoder) i prijemnik (dekode) su neuronske mreže koje se treniraju zajedno, end-to-end, kroz diferencijabilan model kanala. Umesto da inženjer ručno projektuje kodiranje i modulaciju, mreža ih uči samostalno, minimizujući grešku rekonstrukcije poslate poruke.

Ovaj rad opisuje simulaciju takvog sistema na primeru (7,4) autoenkodera, koji preslikava 4 informaciona bita u 7 upotreba kanala (channel uses), što odgovara kodnoj brzini Hamming(7,4) koda. Sistem je najpre realizovan u MATLAB okruženju [2], a zatim nezavisno reprodukovano u PyTorch-u radi unakrsne validacije. Poseban deo posvećen je kvantifikaciji aproksimacija ugrađenih u model, pre svega testiranju naučenog modela na realnijem Rayleigh fading kanalu.

U poglavlju 2 dat je teorijski okvir. Poglavlje 3 opisuje arhitekturu modela i postavku simulacije. Poglavlje 4 prikazuje rezultate MATLAB simulacije i njihovu PyTorch validaciju, a poglavlje 5 analizira aproksimacije i njihov uticaj. Zaključci su dati u poglavlju 6.

2 Teorijska osnova

2.1 Klasičan lanac naspram autoenkodera

Klasičan sistem može se zapisati kao niz transformacija: koder $\rightarrow$ modulator $\rightarrow$ kanal $\rightarrow$ demodulator $\rightarrow$ dekoder. Na primer, Hamming(7,4) kod dodaje tri redundantna bita na svaka četiri informaciona radi detekcije i korekcije grešaka, dok QPSK modulator mapira parove bita u kompleksne simbole. Problem je što su ovi blokovi projektovani nezavisno; optimalan koder i optimalan modulator, spojeni, ne daju nužno optimalan sistem.

Autoenkoder reformuliše problem. Neka je poruka s jedna od $M = 2^k$ mogućih poruka. Enkoder f preslikava poruku, predstavljenu kao one-hot vektor, u n upotreba kanala koje se šalju kroz kanal. Kanal je stohastička transformacija koja simbolu dodaje šum. Dekoder g iz primljenog, izobličenog signala procenjuje koja je poruka poslata. Ceo sistem trenira se da minimizuje verovatnoću pogrešne procene poruke.

2.2 Diferencijabilan kanal i end-to-end trening

Ključni uslov da bi end-to-end trening radio jeste da kanal bude diferencijabilan, kako bi gradijent mogao da se propagira unazad (backpropagation) od dekodera, kroz kanal, do enkodera. AWGN kanal (Additive White Gaussian Noise) ispunjava taj uslov: on prosto dodaje uzorke Gausovog šuma, što je diferencijabilna operacija. Tokom treninga kanal nije parametar koji se uči, već fiksni sloj sa zadatom snagom šuma.

2.3 Ključni parametri

Sistem opisuju sledeći parametri: k je broj informacionih bita po bloku ($k = 4$); n je broj upotreba kanala po bloku ($n = 7$); $M = 2^k$ je broj mogućih poruka ($M = 16$); $R = k/n$ je kodna brzina ($R \approx 0.571$); E_b/N_0 je odnos energije po bitu i spektralne gustine šuma, izražen u dB; a BLER (Block Error Rate) je verovatnoća da je ceo blok pogrešno dekodovan i predstavlja primarnu metriku u ovom radu.

Veza između E_b/N_0 i varijanse šuma po realnoj komponenti data je izrazom:

$$\sigma^2 = 1 / (2R \cdot 10^{(E_b/N_0)dB/10}) \quad (1)$$

Faktor 2 potiče od toga što se snaga deli na realnu i imaginarnu komponentu kompleksnog simbola.

3 Arhitektura modela i postavka simulacije

3.1 Struktura mreže

Model je realizovan kao neuronska mreža tipa autoenkoder. Slika 1 prikazuje strukturu direktno iz MATLAB okruženja: leva tabla daje ceo autoenkoder kao jedinstvenu mrežu, od one-hot ulaza preko sloja kanala do softmax izlaza, dok desne table odvojeno prikazuju enkoder (Tx) i dekoder (Rx) dobijene podelom istrenirane mreže.



Slika 1: Struktura autoenkodera u MATLAB-u — ceo model, enkoder (Tx) i dekodek (Rx).

Enkoder čine dva potpuno povezana sloja (fc_1 i fc_2) sa ReLU aktivacijom između njih (relu_1) i normalizacioni sloj (wnorm). Sloj wnorm odgovara MATLAB-ovom helperAEWNormalizationLayer sloju i ograničava energiju tako da svaki blok ima jediničnu prosečnu snagu po uzorku (Energy normalizacija), a ne po grupi (batch):

$$\tilde{x} = x / \sqrt{(1/n) \sum |x_i|^2} \quad (2)$$

Izlaz enkodera predstavlja $n = 7$ upotreba kanala. Sloj channel je AWGN kanal koji svakom simbolu dodaje Gausov šum sa varijansom datom izrazom (1), pri training $E_b/N_0 = 3$ dB; tokom treninga se ne uči, već je fiksni diferencijabilni sloj. Dekoder prima signal sa izlaza kanala (rxin) i propušta ga kroz fc_3, ReLU (relu_2) i fc_4, pa kroz softmax koji daje raspodelu verovatnoća nad $M = 16$ poruka; procenjena poruka je argmax te raspodele.

Funkcija gubitka je kategorijska unakrsna entropija (categorical cross-entropy):

$$L = - (1/N) \sum \log p_\theta(s | y) \quad (3)$$

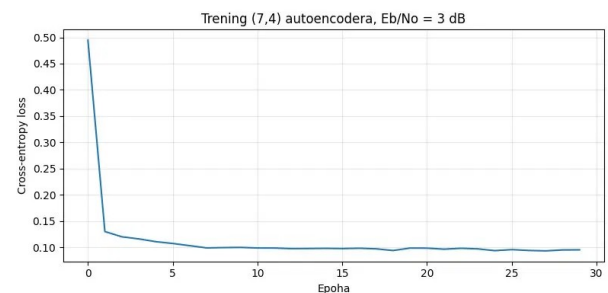
3.2 Baseline sistemi

Da bi se performanse autoenkodera stavile u kontekst, porede se sa tri klasična sistema: (i) nekodovani QPSK, (ii) Hamming(7,4) sa tvrdom odlukom (Hard Decision Decoding, HDD) i (iii) Hamming(7,4) sa dekodovanjem maksimalne verodostojnosti (Maximum Likelihood Decoding, MLD). MLD je optimalan dekodek za dati kod i predstavlja referentnu gornju granicu performansi koju klasičan kod može da postigne.

3.3 Implementacija i trening

Model je prvo realizovan i istreniran u okruženju MATLAB R2023b (Communications Toolbox i Deep Learning Toolbox), a zatim nezavisno reimplementiran u PyTorch-u na platformi Google Colab radi unakrsne validacije. Trening je sproveden Adam optimizatorom uz smanjenje koraka učenja na platou, pri training $E_b/N_0 = 3$ dB.

Slika 2 prikazuje krivu funkcije gubitka tokom treninga PyTorch modela. Gubitak naglo opada već u prvih nekoliko epoha i stabilizuje se na vrednosti oko 0.095, što ukazuje na brzu i stabilnu konvergenciju. Evaluacija BLER krive sprovedena je u opsegu E_b/N_0 od 0 do 8 dB.



Slika 2: Kriva funkcije gubitka tokom treninga ($E_b/N_0 = 3$ dB).

3.4 Slučajni procesi i vođenje simulacije

Simulacija emulira nekoliko slučajnih procesa. Izvor poruka je uniformno slučajan: u svakoj iteraciji treninga i evaluacije bira se niz poruka iz skupa od $M = 16$ mogućih, predstavljenih one-hot vektorima. Šum u kanalu je drugi proces: AWGN sloj svakoj upotrebi kanala dodaje nezavisne uzorke Gausovog šuma nulte

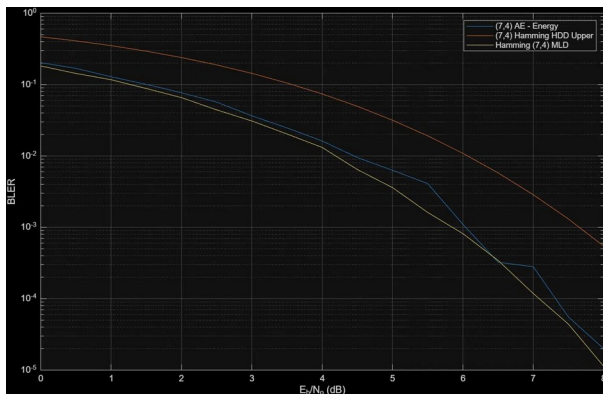
srednje vrednosti i varijanse date izrazom (1). Treći proces, korišćen samo u analizi aproksimacija (poglavlje 5), jeste Rayleigh fading, gde se kompleksni koeficijenti $h \sim \text{CN}(0,1)$ generišu nezavisno po upotrebi kanala. Najzad, inicijalizacija težina mreže je slučajna, pa svaka realizacija treninga polazi iz drugačije početne tačke; to objašnjava sitna odstupanja između MATLAB i PyTorch krivih na Slici 5.

Trening je vođen standardnom petljom po epohama: u svakoj iteraciji generiše se grupa (batch) slučajnih poruka, propušta kroz enkoder, kanal i dekoder, računa se gubitak (3) i ažuriraju težine Adam optimizatorom uz smanjenje koraka učenja na platou; kanal je pri tome fiksiran na $E_b/N_0 = 3$ dB. Evaluacija performansi sprovedena je Monte Carlo metodom: za svaku vrednost E_b/N_0 u opsegu od 0 do 8 dB generiše se velik broj nezavisnih blokova, svaki se propušta kroz sistem pri toj vrednosti E_b/N_0 , a BLER se procenjuje kao odnos broja pogrešno dekodovanih blokova i ukupnog broja poslatih. Najniži pouzdano merljiv BLER time je ograničen brojem uzoraka, pa vrednosti ispod približno 10^{-4} nose statističku nesigurnost, što objašnjava blago neravnomeran rep krivih pri visokim E_b/N_0 . U MATLAB okruženju ceo ovaj tok, od izgradnje mreže preko treninga do evaluacije sva četiri sistema i iscrtavanja, pokreće se jedinstvenim izvršavanjem priloženog Live Script-a.

4 Rezultati

4.1 BLER performanse (MATLAB)

Slika 3 prikazuje BLER krive dobijene MATLAB simulacijom: naučeni autoenkoder sa Energy normalizacijom, Hamming(7,4) sa tvrdom odlukom kao gornja granica, i Hamming(7,4) sa ML dekodovanjem.

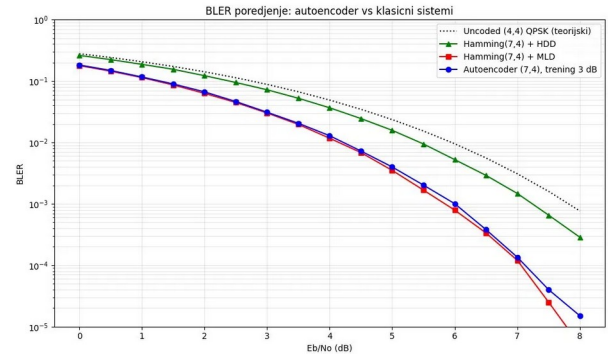


Slika 3: BLER iz MATLAB simulacije (AE Energy, Hamming HDD, Hamming MLD).

Kriva autoenkodera praktično se poklapa sa MLD Hamming dekoderom celom dužinom, dok Hamming sa tvrdom odlukom zaostaje. Na nivou $\text{BLER} = 10^{-2}$, autoenkoderu je potrebno 4.22 dB, a MLD dekoderu 4.15 dB, što daje zanemarljiv gap od svega 0.07 dB. Hamming sa tvrdom odlukom zaostaje za MLD-om približno 1.3 dB. Ovo je centralni nalaz: mreža je, bez ijedne ručno projektovane komponente, samostalno naučila šemu kodiranja i modulacije čije su performanse ekvivalentne optimalnom dekodovanju klasičnog Hamming koda.

4.2 PyTorch reimplementacija i unakrsna validacija

Radi nezavisne provere, isti sistem je reimplementiran u PyTorch-u. Slika 4 prikazuje BLER krive sva četiri sistema, uključujući i nekodovani QPSK kao referentni baseline.



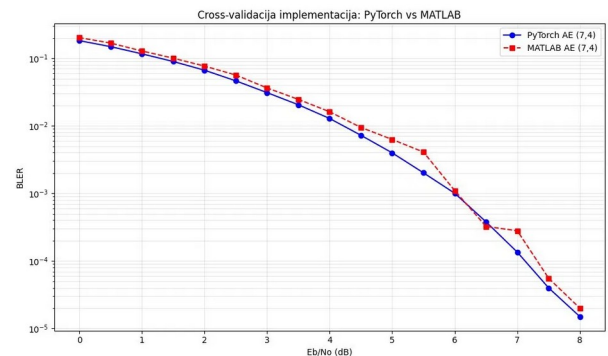
Slika 4: BLER iz PyTorch reimplementacije (sva četiri sistema).

Autoenkoder ostvaruje kodni dobitak (coding gain) od približno 1.73 dB u odnosu na nekodovani QPSK pri $\text{BLER} = 10^{-2}$, a ponovo se poklapa sa MLD Hamming dekoderom. Tabela 1 daje BLER vrednosti očitane na izabranim tačkama.

E_b/N_0	QPSK	HDD	MLD	AE
2 dB	0.12	0.13	0.065	0.068
4 dB	0.046	0.037	0.012	0.013
6 dB	$9.5 \cdot 10^{-3}$	$5.2 \cdot 10^{-3}$	$8 \cdot 10^{-4}$	$1 \cdot 10^{-3}$
8 dB	$7.5 \cdot 10^{-4}$	$2.8 \cdot 10^{-4}$	$2.5 \cdot 10^{-5}$	$1.5 \cdot 10^{-5}$

Tabela 1: BLER na izabranim E_b/N_0 tačkama (očitano sa Slike 4).

Slika 5 direktno preklapa BLER krive autoenkodera iz obe implementacije.



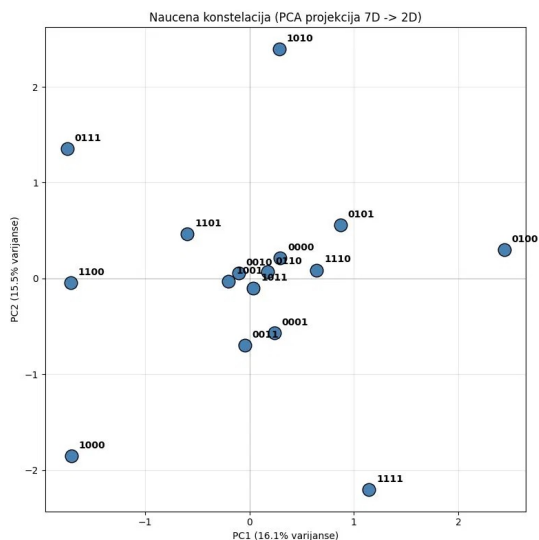
Slika 5: Unakrsna validacija — PyTorch naspram MATLAB autoenkodera.

Krive se poklapaju duž celog opsega; razlike su reda nekoliko desetih delova dB, najizraženije u srednjem opsegu (do približno 0.3 dB), što je očekivano zbog različite inicijalizacije težina i nezavisnih realizacija šuma. Ovo potvrđuje da PyTorch reimplementacija verno reprodukuje MATLAB rezultat.

4.3 Naučena konstelacija

Enkoder preslikava 16 poruka u tačke višedimenzionog prostora (7 upotreba kanala). Radi

vizuelizacije, te tačke su projektovane na 2D ravan metodom analize glavnih komponenti (PCA).



Slika 6: Naučena konstelacija (PCA projekcija u 2D).

Slika 6 pokazuje da naučene tačke nisu raspoređene na standardnoj QPSK ili QAM rešetki; mreža je pronašla sopstveno mapiranje. Prve dve glavne komponente objašnjavaju svega 16.1% i 15.5% varijanse, pa je struktura suštinski višedimenziona i 2D projekcija je samo gruba aproksimacija. MATLAB primer u ekvivalentnoj ulozi koristi t-SNE; PCA je ovde izabran jer je deterministički.

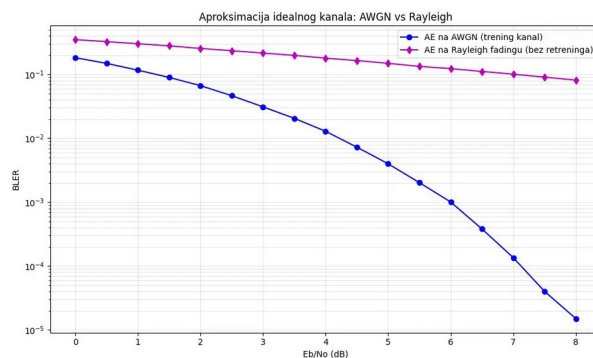
Minimalna Euklidska distanca naučenog koda iznosi približno 3.28, naspram 3.46 za Hamming(7,4) sa BPSK mapiranjem. Iako je minimalna distanca nešto manja, BLER performanse su jednake, jer verovatnoća greške ne zavisi samo od minimalne distance, već od celokupnog spektra distanci i broja najbližih suseda (kissing number).

5 Aproksimacije i njihov uticaj na rezultate

Svaka simulacija počiva na nizu aproksimacija u odnosu na realan sistem. Ovo poglavlje ih identifikuje i, gde je moguće, kvantifikuje njihov uticaj.

5.1 Idealan kanal: AWGN naspram Rayleigh fadinga

Najznačajnija aproksimacija je pretpostavka AWGN kanala. Realni bežični kanal pati od fadinga usled višestaznog prostiranja. Da bi se ova aproksimacija kvantifikovala, naučeni model (treniran isključivo na AWGN) testiran je, bez ikakvog dodatnog treniranja, na ravnom Rayleigh fading kanalu.



Slika 7: Uticaj aproksimacije idealnog kanala — AWGN naspram Rayleigh.

Slika 7 pokazuje rezultat. Na nivou $\text{BLER} = 10^{-1}$, degradacija iznosi približno 5.7 dB. Na nižim nivoima BLER situacija je dramatičnija: model bez retreniranja praktično prestaje da funkcioniše, jer kriva za Rayleigh kanal ne dostiže ni 10^{-2} do 8 dB. Pored pomeranja, menja se i nagib krive: blok fading bez poznavanja stanja kanala (Channel State Information, CSI) ruši red diverziteta na jedinicu, pa BLER opada znatno sporije sa porastom E_b/N_0 . Ovo jasno pokazuje da model naučen na jednom tipu kanala ne generalizuje automatski na drugačiji.

5.2 Ostale aproksimacije

Pored idealnog kanala, model uvodi i niz drugih aproksimacija. Tabela 2 ih sumira zajedno sa procenom uticaja; vrednost označena kao izmerena potiče iz sopstvenog eksperimenta, dok su ostale procene iz literature ili kvalitativne.

#	Aproksimacija	Uticaj	Izvor
1	AWGN kanal (bez fadinga)	≈ 5.7 dB na $\text{BLER} 10^{-1}$	Izmereno (Sl. 7)
2	Savršena sinhronizacija	0.5–2 dB u praksi	Literatura
3	Trening na fiksnom E_b/N_0	sub-optimalnost na ekstremima	Kvalitativno
4	Diskretni model (bez pulse shaping)	zanemaruje ISI	Kvalitativno
5	Float32 vs ADC/DAC rezolucija	0.1–0.5 dB	Literatura
6	Fiksna veličina bloka, bez ARQ	bez retransmisije	Kvalitativno
7	Idealan linearni pojačivač (PAPR)	1–3 dB back-off	Literatura

Tabela 2: Aproksimacije simulacije i procena njihovog uticaja.

6 Zaključak

U ovom radu pokrenut je, analiziran i nezavisno reprodukovan MATLAB primer autoenkodera za bežične komunikacije. MATLAB simulacija je pokazala da (7,4) autoenkoder, treniran end-to-end kroz diferencijabilan AWGN kanal, samostalno uči šemu kodiranja i modulacije čije BLER performanse odgovaraju optimalnom (ML) dekodovanju Hamming(7,4) koda, uz gap od svega 0.07 dB, a uz kodni dobitak od 1.73 dB nad nekodovanim QPSK. PyTorch

reimplementacija je nezavisno potvrdila ovaj rezultat, sa razlikom reda nekoliko desetih delova dB.

Analiza aproksimacija pokazala je da je pretpostavka idealnog AWGN kanala najuticajnija: testiranje na Rayleigh fading kanalu bez retreniranja dovodi do degradacije od približno 5.7 dB na $\text{BLER} = 10^{-1}$ i do gubitka diverziteta na nižim nivoima greške. Ovaj nalaz sugerise pravac daljeg rada: trening na modelu kanala sa fadingom, eventualno uz dostavljanje CSI informacije mreži, kao i ispitivanje robusnosti na nelinearnosti pojačivača i nesavršenu sinhronizaciju. Sav kod, model i interaktivni demo dostupni su javno [5].

Literatura

- [1] T. O'Shea and J. Hoydis, „An Introduction to Deep Learning for the Physical Layer,” IEEE Transactions on Cognitive Communications and Networking, vol. 3, no. 4, pp. 563–575, 2017.
- [2] The MathWorks, Inc., „Autoencoders for Wireless Communications,” MATLAB Communications Toolbox documentation, R2023b. [Online]. Available: <https://www.mathworks.com/help/comm/ug/autoencoders-for-wireless-communications.html>
- [3] R. W. Hamming, „Error detecting and error correcting codes,” The Bell System Technical Journal, vol. 29, no. 2, pp. 147–160, 1950.
- [4] I. Goodfellow, Y. Bengio, and A. Courville, Deep Learning. Cambridge, MA: MIT Press, 2016.
- [5] N. Bakić, „MiSKS Wireless Autoencoder,” GitHub repository, 2026. [Online]. Available: <https://github.com/nikolabakic/misks-wireless-autoencoder>